

Имитационное моделирование

Лабораторная работа №5. Аппарат сетей Петри

Идрисов Джафер Арсенович

Содержание

1	Цель работы	5
2	Задание	6
3	Теоретическое введение	7
3.1	Сети Петри	7
3.2	Задача обедающих философов	8
3.3	Используемые инструменты	8
3.4	Литературное программирование	9
4	Выполнение лабораторной работы	10
4.1	Подготовка окружения	10
4.2	Структура файлов лабораторной работы	11
4.3	Описание модели и модуля DiningPhilosophers.jl	12
4.4	Базовый эксперимент: scripts/dining_philosophers.jl	15
4.5	Анимация процесса: scripts/dining_philosophers_animation.jl	21
4.6	Итоговый сравнительный отчёт: scripts/dining_philosophers_report.jl	23
4.7	Параметрическое исследование: scripts/dining_philosophers_params.jl	27
4.8	Общий вывод по результатам моделирования	32
5	Выводы	33
	Список литературы	34

Список иллюстраций

4.1	Запуск Julia REPL	10
4.2	Подключение пакета DrWatson	10
4.3	Запуск менеджера пакетов и установка зависимостей	11
4.4	Завершение установки пакетов	11
4.5	Запуск основного скрипта <code>dining_philosophers.jl</code>	16
4.6	График эволюции маркировки для классической сети	17
4.7	График эволюции маркировки для сети с арбитром	18
4.8	Содержимое файла <code>dining_classic.csv</code>	18
4.9	Содержимое файла <code>dining_arbiter.csv</code>	19
4.10	Clean-версия основного скрипта	20
4.11	Markdown-версия основного скрипта	20
4.12	Jupyter notebook для основного скрипта	20
4.13	Скрипт построения анимации	21
4.14	Clean-версия скрипта анимации	22
4.15	Markdown-версия скрипта анимации	23
4.16	Notebook-версия скрипта анимации	23
4.17	Скрипт построения итогового сравнительного графика	23
4.18	Сравнительный график состояний <code>Eat_i</code>	25
4.19	Clean-версия скрипта итогового отчёта	26
4.20	Markdown-версия итогового отчёта	26
4.21	Notebook-версия итогового отчёта	26
4.22	Скрипт параметрического исследования	27
4.23	График параметрического исследования	29
4.24	Содержимое файла <code>dining_params.csv</code>	30
4.25	Clean-версия параметрического скрипта	31
4.26	Markdown-версия параметрического исследования	31
4.27	Notebook-версия параметрического исследования	31

Список таблиц

1 Цель работы

Изучить аппарат сетей Петри на примере задачи обедающих философов, реализовать классическую и модифицированную модель на языке Julia, выполнить вычислительные эксперименты, получить графики и таблицы результатов, подготовить literate-версии скриптов и собрать итоговую документацию по лабораторной работе.

2 Задание

1. Создать рабочий каталог и проект Julia в структуре DrWatson.
2. Установить необходимые пакеты для моделирования, визуализации и документирования.
3. Реализовать модель сети Петри для задачи обедающих философов.
4. Выполнить базовый эксперимент для классической сети и сети с арбитром.
5. Построить анимацию изменения маркировки.
6. Сформировать итоговый сравнительный график по состояниям `Eat_i`.
7. Выполнить параметрическое исследование по `N`, `tmax` и `seed`.
8. Подготовить literate-версии скриптов и производные форматы `clean`, `md`, `ipynb`.
9. Описать все полученные графики, CSV-таблицы и листинги кода.

3 Теоретическое введение

3.1 Сети Петри

Сеть Петри представляет собой двудольный ориентированный граф, в котором используются четыре базовых сущности:

- позиции;
- переходы;
- дуги;
- маркировка.

Позиции отражают состояния системы, переходы описывают события, дуги задают причинно-следственные связи, а маркировка определяет текущее распределение фишек по позициям. Формально сеть Петри можно задать четвёркой

$$N = (P, T, F, M_0),$$

где P — множество позиций, T — множество переходов, F — множество дуг, M_0 — начальная маркировка.

Для данной лабораторной работы сеть Петри используется как модель конкурентного доступа к общим ресурсам [7].

3.2 Задача обедающих философов

В классической постановке N философов сидят за круглым столом. Между соседними философами лежит по одной вилке. Чтобы перейти к приёму пищи, философ должен получить две вилки — левую и правую. Если каждый философ одновременно захватывает только одну вилку, система может перейти в состояние взаимной блокировки: все вилки разобраны частично, но никто не может продолжить выполнение.

В терминах сети Петри используются следующие группы позиций:

- `Think_i` — философ i размышляет;
- `Hungry_i` — философ i голоден и ожидает вторую вилку;
- `Eat_i` — философ i ест;
- `Fork_i` — вилка i свободна.

В модификации с арбитром вводится дополнительная позиция `Arbiter c N - 1` фишками. Она запрещает ситуацию, когда все философы одновременно берут по одной вилке, и тем самым устраняет deadlock.

3.3 Используемые инструменты

В работе использовались:

- `Julia` как язык реализации;
- `DrWatson` для организации структуры проекта;
- `OrdinaryDiffEq` для детерминированной формы модели;
- `Plots` для графиков и анимации;
- `CSV` и `DataFrames` для сохранения и анализа результатов;
- `Literate.jl` для генерации `clean, md` и `ipynb` представлений.

Язык `Julia` используется как современная среда для научных вычислений [5]. Для организации воспроизводимого проекта применён `DrWatson` [2], а для

преобразования literate-скриптов в разные представления — пакет `Literate.jl` [3].

3.4 Литературное программирование

Литературное программирование рассматривает программу не только как последовательность инструкций для выполнения, но и как связный текст, объясняющий логику решения. Такой подход был сформулирован Дональдом Кнутом [6] и далее получил развитие в инструментах воспроизводимых вычислений [1].

В данной лабораторной работе этот подход использовался следующим образом:

- основной код каждого эксперимента оформлялся в отдельном `*_literate.jl`-файле;
- из literate-файла генерировались три производных представления:
 - чистый исполняемый `.jl`-скрипт;
 - Markdown-документ `.md`;
 - Jupyter notebook `.ipynb`;
- один и тот же источник одновременно использовался и для вычислений, и для документирования результатов.

Такой способ организации материалов удобен для учебной лабораторной работы, поскольку позволяет избежать расхождения между кодом, текстовым описанием и интерактивной версией эксперимента.

4 Выполнение лабораторной работы

4.1 Подготовка окружения

Сначала был запущен интерпретатор Julia и подготовлен проект лабораторной работы. Далее был подключён пакет DrWatson, активировано окружение проекта и установлены зависимости, используемые в лабораторной работе.

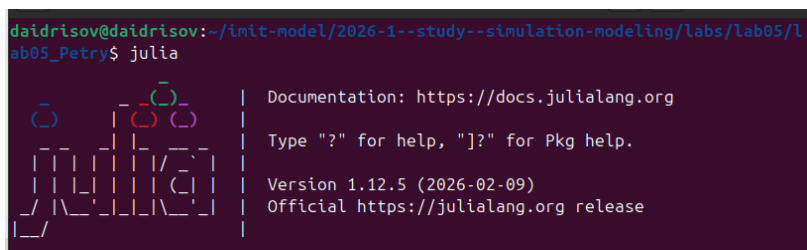
A screenshot of a terminal window showing the Julia REPL startup screen. The prompt is 'daidrisov@daidrisov:~/imit-model/2026-1--study--simulation-modeling/labs/lab05/lab05_Petry\$'. The user has entered 'julia'. The screen displays a stylized logo on the left and text on the right: 'Documentation: https://docs.julialang.org', 'Type "?" for help, "]?" for Pkg help.', 'Version 1.12.5 (2026-02-09)', and 'Official https://julialang.org release'.

Рисунок 4.1: Запуск Julia REPL

На скриншоте показан старт рабочей сессии Julia, с которой начинается настройка окружения и выполнение всех последующих скриптов лабораторной работы.

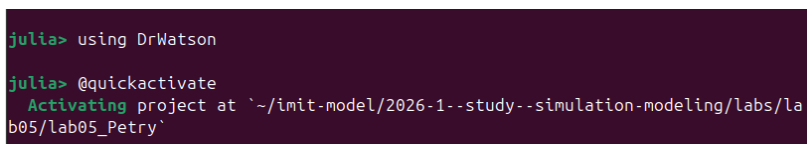
A screenshot of a terminal window showing the Julia REPL session. The prompt is 'julia>'. The user has entered 'using DrWatson'. The prompt is now 'julia> @quickactivate'. The user has entered 'project at `~/imit-model/2026-1--study--simulation-modeling/labs/lab05/lab05\_Petry`'. The screen shows 'Activating project at `~/imit-model/2026-1--study--simulation-modeling/labs/lab05/lab05\_Petry`'.

Рисунок 4.2: Подключение пакета DrWatson

Здесь видно подключение DrWatson, который обеспечивает стандартную структуру проекта и удобные функции для работы с каталогами `src`, `scripts`, `data` и `plots`.

```
julia> Pkg.add([
    "DrWatson",
    "OrdinaryDiffEq",
    "Plots",
    "DataFrames",
    "CSV",
    "Literate",
    "IJulia",
    "Quarto",
])
```

Рисунок 4.3: Запуск менеджера пакетов и установка зависимостей

На этом шаге были подготовлены и добавлены пакеты CSV, DataFrames, DrWatson, IJulia, Literate, OrdinaryDiffEq, Plots и Quarto.

```
Warning: attempting to remove probably stale pidfile 153/155
path = "/home/daidrisov/.julia/compiled/v1.12/OrdinaryDiffEq/DLSvy_ZeWUM.ji.
pidfile"
@ FileWatching.Pidfile /snap/julia/165/share/julia/stdlib/v1.12/FileWatching/s
rc/pidfile.jl:247
Precompiling packages finished.
155 dependencies successfully precompiled in 475 seconds. 254 already precompi
led.
julia> 
```

Рисунок 4.4: Завершение установки пакетов

Скриншот фиксирует успешное завершение подготовки окружения. После этого проект был готов к созданию исходных файлов и запуску экспериментов.

4.2 Структура файлов лабораторной работы

В результате выполнения работы были подготовлены следующие основные файлы:

- `src/DiningPhilosophers.jl` — модуль с описанием структуры сети Петри, моделирования и анализа результатов;
- `scripts/dining_philosophers.jl` — базовый эксперимент для классической сети и сети с арбитром;
- `scripts/dining_philosophers_literate.jl` — literate-версия базового эксперимента;

- `scripts/dining_philosophers_animation.jl` – построение анимации изменения маркировки;
- `scripts/dining_philosophers_animation_literate.jl` – literate-версия скрипта анимации;
- `scripts/dining_philosophers_report.jl` – построение итогового сравнительного графика по состояниям `Eat_i`;
- `scripts/dining_philosophers_report_literate.jl` – literate-версия итогового отчёта;
- `scripts/dining_philosophers_params.jl` – параметрическое исследование;
- `scripts/dining_philosophers_params_literate.jl` – literate-версия параметрического исследования.

4.3 Описание модели и модуля

DiningPhilosophers.jl

Модуль `src/DiningPhilosophers.jl` содержит:

- структуры `PetriNet`;
- функции построения классической сети и сети с арбитром;
- детерминированную модель через систему ОДУ;
- стохастическую симуляцию по схеме Гиллеспи;
- функцию обнаружения deadlock;
- функцию построения графиков эволюции маркировки.

Ключевая идея реализации состоит в том, что матрица инцидентности хранит изменения маркировки для каждой пары «позиция-переход». При стохастическом моделировании из текущей маркировки вычисляются интенсивности допустимых переходов, после чего выбирается очередной переход и обновляется состояние

системы. По сути в скрипте используется событийная схема, близкая к алгоритму Гиллеспи для стохастических систем [4].

4.3.1 Ключевые функции модуля

Ниже приведены основные фрагменты кода модуля `DiningPhilosophers.jl`.

```
struct PetriNet
    n_places::Int
    n_transitions::Int
    incidence::Matrix{Int}
    place_names::Vector{Symbol}
    transition_names::Vector{Symbol}
end

function add_arc!(net::PetriNet, place::Int,
                  transition::Int, sign::Int)
    net.incidence[place, transition] += sign
end
```

```
function build_classical_network(N::Int)
    n_places = 4N
    n_transitions = 3N
    net = PetriNet(n_places, n_transitions)

    for i = 1:N
        net.place_names[i] = Symbol("Think_$i")
        net.place_names[N + i] = Symbol("Hungry_$i")
        net.place_names[2N + i] = Symbol("Eat_$i")
        net.place_names[3N + i] = Symbol("Fork_$i")
    end
```

```

    end
    # ...
    return net, u0, net.place_names
end

```

```

function detect_deadlock(df::DataFrame, net::PetriNet;
                        tol = 1e-6)
    u_last = [df[end, String(net.place_names[i])]]
    for i = 1:net.n_places]
    for j = 1:net.n_transitions
        can_fire = true
        for i = 1:net.n_places
            if net.incidence[i, j] < 0 &&
                u_last[i] < -net.incidence[i, j] - tol
                can_fire = false
                break
            end
        end
        if can_fire
            return false
        end
    end
    return true
end

```

Эти функции определяют структуру сети, построение маркировки и критерий взаимной блокировки.

4.4 Базовый эксперимент:

scripts/dining_philosophers.jl

Основной скрипт лабораторной работы запускает две модели:

- классическую сеть без арбитра;
- сеть с арбитром.

Скрипт создаёт сети, запускает стохастическую симуляцию, сохраняет траектории в CSV, проверяет наличие deadlock и строит графики эволюции маркировки.

4.4.1 Ключевой фрагмент кода

```
function main()
    N = 5
    tmax = 50.0

    net_classic, u0_classic, _ =
        DiningPhilosophers.build_classical_network(N)
    df_classic =
        DiningPhilosophers.simulate_stochastic(
            net_classic, u0_classic, tmax
        )
    CSV.write(datadir("dining_classic.csv"), df_classic)

    dead =
        DiningPhilosophers.detect_deadlock(
            df_classic, net_classic
        )
```

```

plot_classic =
    DiningPhilosophers.plot_marking_evolution(
        df_classic, N
    )
savefig(plot_classic,
        plotsdir("classic_simulation.png"))
end

```

Этот фрагмент показывает общий шаблон базового эксперимента: построение сети, моделирование, проверку deadlock и сохранение результатов.

```

julia> include("scripts/dining_philosophers.jl")
=== Классическая сеть (без арбитра) ===
Deadlock обнаружен: true

=== Сеть с арбитром ===
Could not create decoration from factory! Running with no decorations.
Deadlock обнаружен: false
"/home/daidrisov/imit-model/2026-1--study--simulation-modeling/labs/lab05/lab05_
Petry/plots/arbiter_simulation.png"

```

Рисунок 4.5: Запуск основного скрипта `dining_philosophers.jl`

Этот скриншот подтверждает выполнение базового сценария. На его основе формируются два ключевых файла данных: `dining_classic.csv` и `dining_arbiter.csv`.

4.4.2 Результаты для классической сети

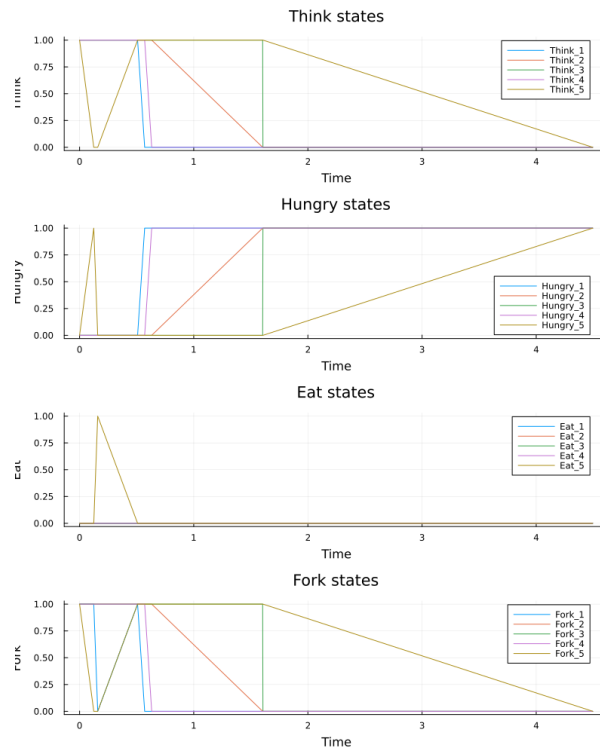


Рисунок 4.6: График эволюции маркировки для классической сети

На графике показана динамика четырёх групп состояний:

- $Think_i$ — фишка покидает позицию после начала захвата вилок;
- $Hungry_i$ — число голодных философов постепенно растёт;
- Eat_i — состояние приёма пищи оказывается кратковременным;
- $Fork_i$ — свободные вилки исчезают по мере захвата ресурсов.

Финальная часть графика соответствует взаимной блокировке: философы перестают переходить в состояние Eat , а вилки становятся недоступны. По содержимому последней строки `dining_classic.csv` видно, что в момент $t \approx 6.62$ все пять философов находятся в состоянии $Hungry$, все $Eat_i = 0$, а все $Fork_i = 0$. Это и есть *deadlock*.

4.4.3 Результаты для сети с арбитром

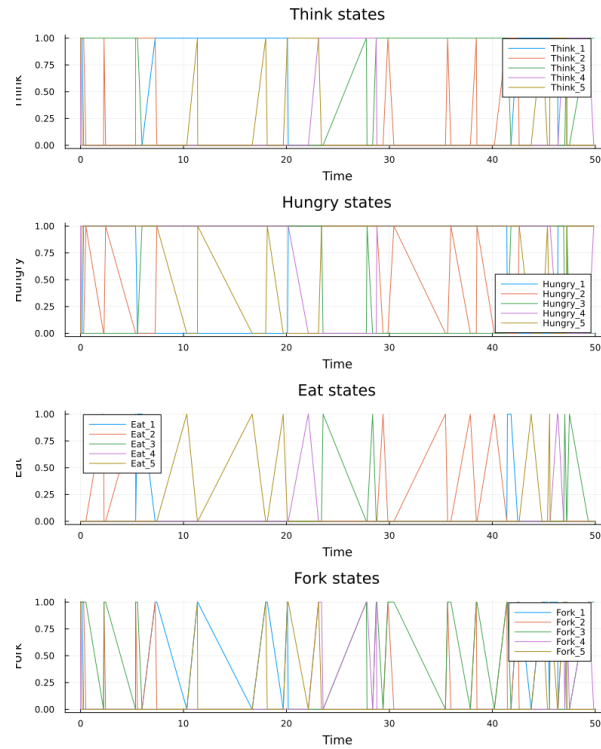


Рисунок 4.7: График эволюции маркировки для сети с арбитром

На этом графике система ведёт себя иначе. Несмотря на постоянную конкуренцию за ресурсы, переходы продолжают срабатывать до конца моделирования. На интервале до $t \approx 49$ наблюдается чередование состояний Think, Hungry и Eat, а позиция Arbiter ограничивает число одновременно пытающихся поесть философов. Deadlock не возникает.

4.4.4 CSV-таблица dining_classic.csv

	time	Think_1	Think_2	Think_3	Think_4	Think_5	Hungry_1	Hungry_2	Hungry_3	Hungry_4	Hungry_5	Eat_1	Eat_2	Eat_3	Eat_4	Eat_5	Fork_1	Fork_2	Fork_3	Fork_4	Fork_5
1	0.0	1.0	1.0	1.0	1.0	1.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	1.0	1.0	1.0	1.0	1.0
2	0.1407970600777255	1.0	1.0	1.0	0.0	1.0	0.0	0.0	0.0	1.0	0.0	0.0	0.0	0.0	0.0	0.0	1.0	1.0	1.0	0.0	1.0
3	0.45150760243165033	1.0	1.0	1.0	0.0	0.0	0.0	0.0	0.0	1.0	0.0	0.0	0.0	0.0	0.0	0.0	1.0	1.0	1.0	0.0	0.0
4	0.4623109404407648	1.0	1.0	0.0	0.0	0.0	0.0	0.0	1.0	1.0	1.0	0.0	0.0	0.0	0.0	0.0	1.0	1.0	0.0	0.0	0.0
5	1.3175013653007066	1.0	1.0	0.0	0.0	0.0	0.0	0.0	1.0	1.0	0.0	0.0	0.0	0.0	0.0	1.0	0.0	1.0	0.0	0.0	0.0
6	2.67510161742008	1.0	0.0	0.0	0.0	0.0	0.0	1.0	1.0	1.0	0.0	0.0	0.0	0.0	0.0	1.0	0.0	0.0	0.0	0.0	0.0
7	4.0032715877257234	1.0	0.0	0.0	0.0	1.0	0.0	1.0	1.0	1.0	0.0	0.0	0.0	0.0	0.0	0.0	1.0	0.0	0.0	0.0	1.0
8	4.110862935540009	1.0	0.0	0.0	0.0	1.0	0.0	1.0	1.0	0.0	0.0	0.0	0.0	0.0	1.0	0.0	1.0	0.0	0.0	0.0	0.0
9	4.55339478870931	0.0	0.0	0.0	0.0	1.0	1.0	1.0	1.0	0.0	0.0	0.0	0.0	0.0	1.0	0.0	0.0	0.0	0.0	0.0	0.0
10	5.340941915174316	0.0	0.0	0.0	1.0	1.0	1.0	1.0	1.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	1.0	1.0
11	5.8125820602324	0.0	0.0	0.0	0.0	1.0	1.0	1.0	1.0	1.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	1.0
12	6.619501244190362	0.0	0.0	0.0	0.0	0.0	1.0	1.0	1.0	1.0	1.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0

Рисунок 4.8: Содержимое файла dining_classic.csv

Файл `dining_classic.csv` содержит полную временную траекторию классической сети. Столбцы интерпретируются так:

- `time` — модельное время;
- `Think_1 ... Think_5` — наличие философа в состоянии размышления;
- `Hungry_1 ... Hungry_5` — наличие философа в состоянии ожидания;
- `Eat_1 ... Eat_5` — наличие философа в состоянии приёма пищи;
- `Fork_1 ... Fork_5` — наличие свободных вилок.

Каждый из этих столбцов фактически содержит число фишек в соответствующей позиции. В данной постановке значения обычно равны 0 или 1, поскольку философ не может одновременно находиться в нескольких состояниях.

4.4.5 CSV-таблица `dining_arbiter.csv`

	time	Think_1	Think_2	Think_3	Think_4	Think_5	Hungry_1	Hungry_2	Hungry_3	Hungry_4	Hungry_5	Eat_1	Eat_2	Eat_3	Eat_4	Eat_5	Fork_1	Fork_2	Fork_3	Fork_4	Fork_5	Arbiter
1	0.0	1.0	1.0	1.0	1.0	1.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	1.0	1.0	1.0	1.0	1.0	4.0
2	0.0991106242396261	1.0	1.0	1.0	0.0	1.0	0.0	0.0	0.0	1.0	0.0	0.0	0.0	0.0	0.0	0.0	1.0	1.0	1.0	0.0	1.0	3.0
3	0.12313518716467611	0.0	1.0	1.0	0.0	1.0	1.0	0.0	0.0	1.0	0.0	0.0	0.0	0.0	0.0	0.0	1.0	1.0	0.0	1.0	2.0	
4	0.21547129933679924	0.0	1.0	0.0	0.0	1.0	1.0	0.0	1.0	1.0	0.0	0.0	0.0	0.0	0.0	0.0	1.0	0.0	0.0	1.0	1.0	
5	0.5294687978949245	0.0	1.0	0.0	0.0	1.0	1.0	0.0	1.0	0.0	0.0	0.0	0.0	0.0	1.0	0.0	0.0	1.0	0.0	0.0	0.0	1.0
6	1.1519948218902419	0.0	1.0	0.0	1.0	1.0	0.0	0.0	1.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	1.0	0.0	1.0	1.0	2.0	
7	1.3772096245222528	0.0	1.0	0.0	0.0	1.0	1.0	0.0	1.0	1.0	0.0	0.0	0.0	0.0	0.0	0.0	1.0	0.0	0.0	1.0	1.0	
8	2.0774794515152762	0.0	0.0	0.0	0.0	1.0	1.0	1.0	1.0	1.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	1.0	0.0	
9	2.5623699575823866	0.0	0.0	0.0	0.0	1.0	1.0	1.0	1.0	0.0	0.0	0.0	0.0	0.0	1.0	0.0	0.0	0.0	0.0	0.0	0.0	

Рисунок 4.9: Содержимое файла `dining_arbiter.csv`

Структура файла `dining_arbiter.csv` совпадает с `dining_classic.csv`, но дополнительно содержит столбец:

- `Arbiter` — число фишек в позиции арбитра.

Если `Arbiter > 0`, система допускает новый вход философа в критическую секцию захвата ресурсов. Когда фишка арбитра временно отсутствует, очередной философ не может начать цикл захвата вилок. Именно этот дополнительный ресурс устраняет тупиковую конфигурацию.

4.4.6 Производные форматы для основного скрипта

После подготовки `literate`-версии были сгенерированы производные представления.

```
julia> Literate.script("scripts/dining_philosophers.jl", "scripts"; name="dining_
philosophers_clean")
[ Info: generating plain script file from `~/imit-model/2026-1--study--simulation-modeling/labs/lab05/lab05_Petry/scripts/dining_philosophers.jl`
[ Info: writing result to `~/imit-model/2026-1--study--simulation-modeling/labs/lab05/lab05_Petry/scripts/dining_philosophers_clean.jl`
"/home/daidrisov/imit-model/2026-1--study--simulation-modeling/labs/lab05/lab05_Petry/scripts/dining_philosophers_clean.jl"
```

Рисунок 4.10: Clean-версия основного скрипта

Скриншот показывает автоматически сформированный `clean`-скрипт, пригодный для прямого запуска без `literate`-разметки.

```
julia> Literate.markdown("scripts/dining_philosophers.jl", "docs"; name="dining_
philosophers", execute=true)
[ Info: generating markdown page from `~/imit-model/2026-1--study--simulation-modeling/labs/lab05/lab05_Petry/scripts/dining_philosophers.jl`
[ Info: writing result to `~/imit-model/2026-1--study--simulation-modeling/labs/lab05/lab05_Petry/docs/dining_philosophers.md`
"/home/daidrisov/imit-model/2026-1--study--simulation-modeling/labs/lab05/lab05_Petry/docs/dining_philosophers.md"
```

Рисунок 4.11: Markdown-версия основного скрипта

Файл `.md` использовался как текстовая документация и как источник для дальнейшего включения материала в отчёт.

```
julia> Literate.notebook("scripts/dining_philosophers.jl", "notebook"; name="din
ing_philosophers", execute=true)
[ Info: generating notebook from `~/imit-model/2026-1--study--simulation-modeling/labs/lab05/lab05_Petry/scripts/dining_philosophers.jl`
[ Info: executing notebook `dining_philosophers.ipynb`
[ Info: writing result to `~/imit-model/2026-1--study--simulation-modeling/labs/lab05/lab05_Petry/notebook/dining_philosophers.ipynb`
"/home/daidrisov/imit-model/2026-1--study--simulation-modeling/labs/lab05/lab05_Petry/notebook/dining_philosophers.ipynb"
```

Рисунок 4.12: Jupyter notebook для основного скрипта

Notebook-версия позволила выполнить тот же эксперимент в интерактивной форме и визуально проверить воспроизводимость результата.

4.5 Анимация процесса:

`scripts/dining_philosophers_animation.jl`

Следующий скрипт строит анимацию изменения маркировки классической сети для малого числа философов $N = 3$.

```
julia> include("scripts/dining_philosophers_animation.jl")
Could not create decoration from factory! Running with no decorations.
[ Info: Saved animation to /home/daidrisov/imit-model/2026-1--study--simulation-
modeling/labs/lab05/lab05_Petry/plots/philosophers_simulation.gif
Анимация сохранена в plots/philosophers_simulation.gif
```

Рисунок 4.13: Скрипт построения анимации

Логика скрипта состоит в следующем:

- создаётся классическая сеть Петри;
- выполняется стохастическая симуляция;
- для каждой строки таблицы создаётся кадр столбчатой диаграммы;
- кадры сохраняются в GIF-файл `philosophers_simulation.gif`.

Эта анимация особенно полезна тем, что позволяет увидеть не только итоговое состояние, но и весь процесс постепенного перехода сети к блокировке.

4.5.1 Ключевой фрагмент кода

```
function main()
    N = 3
    tmax = 30.0
    net, u0, names =
        DiningPhilosophers.build_classical_network(N)

    Random.seed!(123)
    df =
```

```

        DiningPhilosophers.simulate_stochastic(
            net, u0, tmax
        )

    anim = @animate for row in eachrow(df)
        u = [row[col] for col in propertynames(row)
            if col != :time]
        bar(1:length(u), u, legend = false)
        xticks!(1:length(u), string.(names),
            rotation = 45)
    end

    gif(anim, plotsdir("philosophers_simulation.gif"),
        fps = 2)
end

```

Код анимации строит последовательность кадров по строкам таблицы состояний и сохраняет GIF-файл.

4.5.2 Производные форматы для скрипта анимации

```

julia> Literate.script("scripts/dining_philosophers_animation.jl", "scripts"; na
me="dining_philosophers_animation_clean")
[ Info: generating plain script file from `~/imit-model/2026-1--study--simulatio
n-modeling/labs/lab05/lab05_Petry/scripts/dining_philosophers_animation.jl`
[ Info: writing result to `~/imit-model/2026-1--study--simulation-modeling/labs/
lab05/lab05_Petry/scripts/dining_philosophers_animation_clean.jl`
~/home/daidrisov/imit-model/2026-1--study--simulation-modeling/labs/lab05/lab05_
Petry/scripts/dining_philosophers_animation_clean.jl"

```

Рисунок 4.14: Clean-версия скрипта анимации

Clean-представление содержит только исполняемый код и используется как конечный вариант сценария генерации GIF.

```
julia> Literate.markdown("scripts/dining_philosophers_animation.jl", "docs"; name="dining_philosophers_animation", execute=true)
[ Info: generating markdown page from `~/imit-model/2026-1--study--simulation-modeling/labs/lab05/lab05_Petry/scripts/dining_philosophers_animation.jl`
[ Info: writing result to `~/imit-model/2026-1--study--simulation-modeling/labs/lab05/lab05_Petry/docs/dining_philosophers_animation.md`
~/home/daidrisov/imit-model/2026-1--study--simulation-modeling/labs/lab05/lab05_Petry/docs/dining_philosophers_animation.md"
```

Рисунок 4.15: Markdown-версия скрипта анимации

Markdown-файл фиксирует словесное описание анимационного сценария и последовательность построения кадров.

```
julia> Literate.notebook("scripts/dining_philosophers_animation.jl", "notebook"; name="dining_philosophers_animation", execute=true)
[ Info: generating notebook from `~/imit-model/2026-1--study--simulation-modeling/labs/lab05/lab05_Petry/scripts/dining_philosophers_animation.jl`
[ Info: executing notebook `dining_philosophers_animation.ipynb`
[ Info: writing result to `~/imit-model/2026-1--study--simulation-modeling/labs/lab05/lab05_Petry/notebook/dining_philosophers_animation.ipynb`
~/home/daidrisov/imit-model/2026-1--study--simulation-modeling/labs/lab05/lab05_Petry/notebook/dining_philosophers_animation.ipynb"
```

Рисунок 4.16: Notebook-версия скрипта анимации

Notebook-версия удобна для визуального просмотра построения анимации и проверки каждого шага отдельно.

4.6 Итоговый сравнительный отчёт:

scripts/dining_philosophers_report.jl

Скрипт `dining_philosophers_report.jl` загружает данные базового эксперимента и строит сводный график по состояниям `Eat_i`.

```
julia> include("scripts/dining_philosophers_report.jl")
Отчёт сохранён в plots/final_report.png
```

Рисунок 4.17: Скрипт построения итогового сравнительного графика

Основная задача этого файла — не моделирование, а повторная обработка уже полученных CSV-таблиц.

4.6.1 Ключевой фрагмент кода

```
function main()
    df_classic =
        CSV.read(datadir("dining_classic.csv"),
                  DataFrame)
    df_arbiter =
        CSV.read(datadir("dining_arbiter.csv"),
                  DataFrame)

    N = 5

    eat_cols = [Symbol("Eat_$i") for i = 1:N]

    p1 = plot(df_classic.time,
              Matrix(df_classic[:, eat_cols]),
              title = "XXXXXXXXXXXX XXXX")
    p2 = plot(df_arbiter.time,
              Matrix(df_arbiter[:, eat_cols]),
              title = "XXXX X XXXXXXXXXX")

    p_final = plot(p1, p2, layout = (2, 1),
                   size = (800, 600))
    savefig(p_final, plotsdir("final_report.png"))
end
```

В этом фрагменте показана повторная загрузка экспериментальных данных и построение сводного графика для двух сценариев.

4.6.2 График `final_report.png`

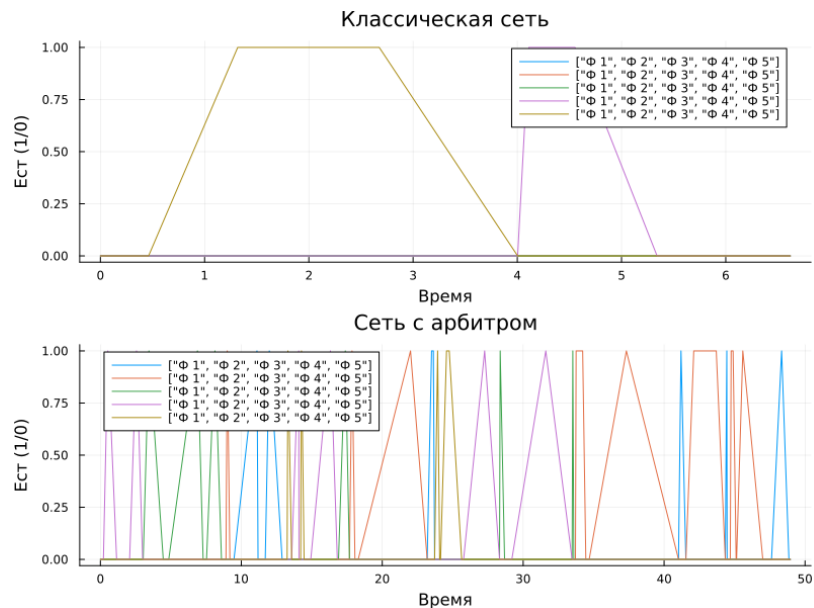


Рисунок 4.18: Сравнительный график состояний Eat_i

На верхней панели графика показана классическая сеть. Все кривые Eat_i довольно быстро опускаются к нулю и дальше остаются на нуле. Это означает, что философы перестали получать возможность есть.

На нижней панели показана сеть с арбитром. Здесь линии колеблются на всём интервале времени, а значит система не теряет живость и хотя бы часть философов продолжает получать доступ к ресурсам.

Именно этот график наиболее наглядно демонстрирует различие между двумя постановками задачи:

- в классической сети deadlock возникает неизбежно;
- в сети с арбитром приём пищи продолжается на протяжении моделирования.

4.6.3 Производные форматы для итогового отчёта

```
julia> Literate.script("scripts/dining_philosophers_report.jl", "scripts"; name="dining_philosophers_report_clean")
[ Info: generating plain script file from `~/imit-model/2026-1--study--simulation-modeling/labs/lab05/lab05_Petry/scripts/dining_philosophers_report.jl`
[ Info: writing result to `~/imit-model/2026-1--study--simulation-modeling/labs/lab05/lab05_Petry/scripts/dining_philosophers_report_clean.jl`
"/home/daidrisov/imit-model/2026-1--study--simulation-modeling/labs/lab05/lab05_Petry/scripts/dining_philosophers_report_clean.jl"
```

Рисунок 4.19: Clean-версия скрипта итогового отчёта

Clean-версия фиксирует исполняемый код для построения итогового графика без literate-комментариев.

```
julia> Literate.markdown("scripts/dining_philosophers_report.jl", "docs"; name="dining_philosophers_report", execute=true)
[ Info: generating markdown page from `~/imit-model/2026-1--study--simulation-modeling/labs/lab05/lab05_Petry/scripts/dining_philosophers_report.jl`
[ Info: writing result to `~/imit-model/2026-1--study--simulation-modeling/labs/lab05/lab05_Petry/docs/dining_philosophers_report.md`
"/home/daidrisov/imit-model/2026-1--study--simulation-modeling/labs/lab05/lab05_Petry/docs/dining_philosophers_report.md"
```

Рисунок 4.20: Markdown-версия итогового отчёта

В Markdown-представлении содержится пояснение к идее сравнения состояний Eat_i и интерпретация графика.

```
julia> Literate.notebook("scripts/dining_philosophers_report.jl", "notebook"; name="dining_philosophers_report", execute=true)
[ Info: generating notebook from `~/imit-model/2026-1--study--simulation-modeling/labs/lab05/lab05_Petry/scripts/dining_philosophers_report.jl`
[ Info: executing notebook `dining_philosophers_report.ipynb`
[ Info: writing result to `~/imit-model/2026-1--study--simulation-modeling/labs/lab05/lab05_Petry/notebook/dining_philosophers_report.ipynb`
"/home/daidrisov/imit-model/2026-1--study--simulation-modeling/labs/lab05/lab05_Petry/notebook/dining_philosophers_report.ipynb"
```

Рисунок 4.21: Notebook-версия итогового отчёта

Notebook-версия обеспечивает интерактивный просмотр итогового анализа и повторное построение графика.

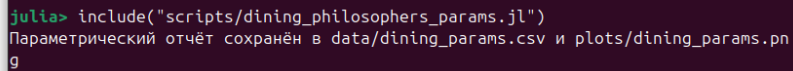
4.7 Параметрическое исследование:

`scripts/dining_philosophers_params.jl`

Параметрический скрипт выполняет серию запусков для наборов параметров:

- `N = [3, 5, 7];`
- `tmax = [30.0, 50.0, 80.0];`
- `seed = [123, 124, 125].`

При каждом запуске рассматриваются две сети — `classic` и `arbiter`. Для них сохраняются признаки наличия deadlock, число событий и значения `final_hungry`, `final_eat`.



```
julia> include("scripts/dining_philosophers_params.jl")
Параметрический отчёт сохранён в data/dining_params.csv и plots/dining_params.png
```

Рисунок 4.22: Скрипт параметрического исследования

Этот файл агрегирует результаты сразу по 54 прогонам: 27 комбинаций параметров для классической сети и 27 — для сети с арбитром.

4.7.1 Ключевой фрагмент кода

```
function main()
    N_VALUES = [3, 5, 7]
    TMAX_VALUES = [30.0, 50.0, 80.0]
    SEEDS = [123, 124, 125]

    results = DataFrame(
        network = String[],
        N = Int[],
```

```

    tmax = Float64[],
    seed = Int[],
    deadlock = Bool[],
    events = Int[],
    final_hungry = Float64[],
    final_eat = Float64[],
)

for N in N_VALUES, tmax in TMAX_VALUES, seed in SEEDS
    # XXXXX classical X arbiter
    # XXXXXXXXXXXX XXXXXXX results
end

CSV.write(datadir("dining_params.csv"), results)
end

```

В параметрическом скрипте центральным элементом является тройной цикл по N, tmax и seed, после которого агрегированные результаты сохраняются в CSV и визуализируются на отдельном графике.

4.7.2 График `dining_params.png`

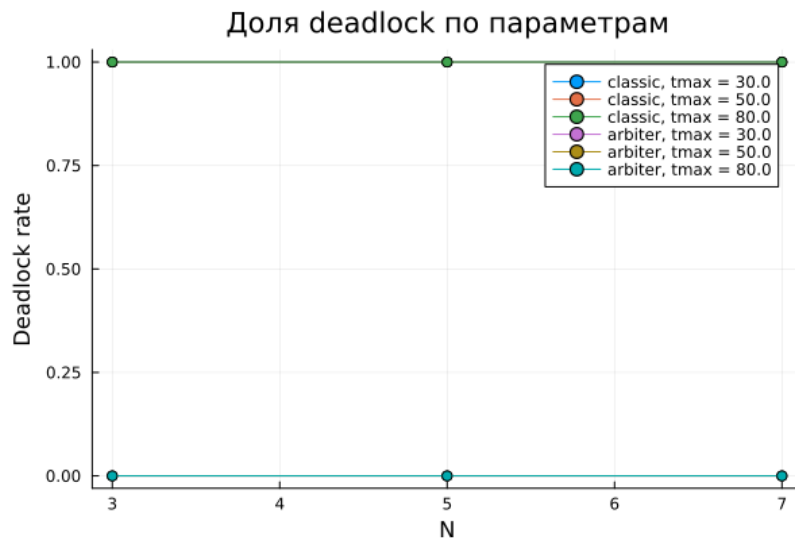


Рисунок 4.23: График параметрического исследования

На графике по оси `N` отложено число философов, а по оси `Deadlock rate` — доля прогонов, завершившихся взаимной блокировкой.

Фактический результат оказался очень устойчивым:

- для сети `classic` `deadlock_rate` = 1 при всех значениях `N` и `tmax`;
- для сети `arbiter` `deadlock_rate` = 0 при всех значениях `N` и `tmax`.

Дополнительно по `dining_params.csv` можно сделать ещё два вывода:

- среднее число событий в классической сети равно примерно 17.67, то есть система быстро приходит к блокировке;
- среднее число событий в сети с арбитром равно примерно 93.33, что отражает существенно более длительную активную работу системы.

4.7.3 CSV-таблица `dining_params.csv`

	network	N	tmax	seed	deadlock	events	final_hungry	final_eat
1	classic	3	30.0	123	true	7	3.0	0.0
2	arbiter	3	30.0	123	false	51	2.0	0.0
3	classic	3	30.0	124	true	7	3.0	0.0
4	arbiter	3	30.0	124	false	57	2.0	0.0
5	classic	3	30.0	125	true	4	3.0	0.0
6	arbiter	3	30.0	125	false	45	2.0	0.0
7	classic	3	50.0	123	true	7	3.0	0.0
8	arbiter	3	50.0	123	false	82	1.0	1.0
9	classic	3	50.0	124	true	7	3.0	0.0
10	arbiter	3	50.0	124	false	77	1.0	0.0
11	classic	3	50.0	125	true	4	3.0	0.0
12	arbiter	3	50.0	125	false	85	1.0	1.0
13	classic	3	80.0	123	true	7	3.0	0.0

Рисунок 4.24: Содержимое файла `dining_params.csv`

Файл `dining_params.csv` содержит агрегированную информацию о каждом прогоне. Столбцы имеют следующий смысл:

- `network` — тип сети: `classic` или `arbiter`;
- `N` — число философов;
- `tmax` — предельное время моделирования;
- `seed` — зерно генератора случайных чисел;
- `deadlock` — булев признак наличия `deadlock` в конце прогона;
- `events` — число зафиксированных состояний в траектории;
- `final_hungry` — суммарное число философов в состоянии `Hungry` в последней строке;
- `final_eat` — суммарное число философов в состоянии `Eat` в последней строке.

Из этой таблицы видно, что:

- в строках сети `classic` признак `deadlock` всегда равен `true`;
- в строках сети `arbiter` признак `deadlock` всегда равен `false`;
- для классической сети `final_eat` во всех прогонах равен нулю;

- для сети с арбитром в конце моделирования нередко сохраняется хотя бы один философ в состоянии приёма пищи.

4.7.4 Производные форматы для параметрического исследования

```
julia> Literate.script("scripts/dining_philosophers_params.jl", "scripts"; name="dining_philosophers_params_clean")
[ Info: generating plain script file from `~/imit-model/2026-1--study--simulation-modeling/labs/lab05/lab05_Petry/scripts/dining_philosophers_params.jl`
[ Info: writing result to `~/imit-model/2026-1--study--simulation-modeling/labs/lab05/lab05_Petry/scripts/dining_philosophers_params_clean.jl`
~/home/daidrisov/imit-model/2026-1--study--simulation-modeling/labs/lab05/lab05_Petry/scripts/dining_philosophers_params_clean.jl"
```

Рисунок 4.25: Clean-версия параметрического скрипта

Clean-версия содержит исполняемый сценарий многократного прогона без разметки `literate programming`.

```
julia> Literate.markdown("scripts/dining_philosophers_params.jl", "docs"; name="dining_philosophers_params", execute=true)
[ Info: generating markdown page from `~/imit-model/2026-1--study--simulation-modeling/labs/lab05/lab05_Petry/scripts/dining_philosophers_params.jl`
[ Info: writing result to `~/imit-model/2026-1--study--simulation-modeling/labs/lab05/lab05_Petry/docs/dining_philosophers_params.md`
~/home/daidrisov/imit-model/2026-1--study--simulation-modeling/labs/lab05/lab05_Petry/docs/dining_philosophers_params.md"
```

Рисунок 4.26: Markdown-версия параметрического исследования

Markdown-файл удобен как краткая текстовая документация для параметрического анализа.

```
julia> Literate.notebook("scripts/dining_philosophers_params.jl", "notebook"; name="dining_philosophers_params", execute=true)
[ Info: generating notebook from `~/imit-model/2026-1--study--simulation-modeling/labs/lab05/lab05_Petry/scripts/dining_philosophers_params.jl`
[ Info: executing notebook `dining_philosophers_params.ipynb`
[ Info: writing result to `~/imit-model/2026-1--study--simulation-modeling/labs/lab05/lab05_Petry/notebook/dining_philosophers_params.ipynb`
~/home/daidrisov/imit-model/2026-1--study--simulation-modeling/labs/lab05/lab05_Petry/notebook/dining_philosophers_params.ipynb"
```

Рисунок 4.27: Notebook-версия параметрического исследования

Notebook-версия позволяет выполнить серию запусков в интерактивной форме и пересчитать график при изменении параметров.

4.8 Общий вывод по результатам моделирования

Полученные файлы данных и графики согласуются между собой:

- `dining_classic.csv` показывает быстрое вырождение траектории в `dead-lock`;
- `dining_arbiter.csv` показывает, что система остаётся активной до конца интервала моделирования;
- `final_report.png` количественно подтверждает исчезновение состояний `Eat_i` в классической сети;
- `dining_params.csv` и `dining_params.png` показывают, что результат не зависит от выбранных `N`, `tmax` и `seed`: классическая сеть всегда тупиковая, сеть с арбитром — нет.

5 Выводы

В ходе лабораторной работы был реализован и исследован аппарат сетей Петри на примере задачи обедающих философов. Построены две модели: классическая и модифицированная с арбитром. Базовый эксперимент показал, что классическая постановка быстро приводит к deadlock, тогда как введение арбитра сохраняет активность системы на всём интервале моделирования.

Были получены:

- графики эволюции маркировки для обеих сетей;
- CSV-файлы с полными траекториями состояний;
- анимация изменения маркировки;
- итоговый сравнительный график по состояниям `Eat_i`;
- таблица и график параметрического исследования;
- `iterate`-версии скриптов и производные форматы `clean`, `md`, `ipynb`.

Параметрический анализ подтвердил устойчивость результата: для всех 27 комбинаций параметров классическая сеть завершалась deadlock, а сеть с арбитром не блокировалась ни в одном запуске. Следовательно, введение дополнительного механизма синхронизации в виде арбитра является эффективным способом устранения взаимной блокировки в задаче обедающих философов.

Список литературы

1. A Multi-Language Computing Environment for Literate Programming and Reproducible Research / E. Schulte [et al.] // Journal of Statistical Software. — 2012. — Vol. 46, no. 3. — P. 1–24. — DOI: 10.18637/jss.v046.i03.
2. *Datseris G., contributors.* DrWatson.jl Documentation. — 2024. — URL: <https://juliadynamics.github.io/DrWatson.jl/stable/> (visited on 04/12/2026).
3. *Ekre F., contributors.* Literate.jl Documentation. — 2024. — URL: <https://fredrikekre.github.io/Literate.jl/v2/> (visited on 04/12/2026).
4. *Gillespie D. T.* Exact Stochastic Simulation of Coupled Chemical Reactions // The Journal of Physical Chemistry. — 1977. — Vol. 81, no. 25. — P. 2340–2361. — DOI: 10.1021/j100540a008.
5. Julia: A Fresh Approach to Numerical Computing / J. Bezanson [et al.] // SIAM Review. — 2017. — Vol. 59, no. 1. — P. 65–98. — DOI: 10.1137/141000671.
6. *Knuth D. E.* Literate Programming // The Computer Journal. — 1984. — Vol. 27, no. 2. — P. 97–111. — DOI: 10.1093/comjnl/27.2.97.
7. *Peterson J. L.* Petri Net Theory and the Modeling of Systems. — Englewood Cliffs, NJ : Prentice Hall, 1981.